

MPI Distributed Programming Examples

MPI makes use of the distributed programming paradigm, also known as the "message-passing" approach.

Our examples are written as C source code, hence, the usage in Colab will be slightly different than the one when using straight Python source code.

First, the code will have to be written into text files, then the source file will be compiled, and finally the executable file created from the compilation will be run.

✓ Compiling and running MPI Hello World Example

```
%%sh
cat > hello_rank.c << EOF
/*
 * Exemplo retirado da Internet
 */

#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {

    MPI_Init(NULL, NULL);

    int rank;
    int size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello: rank %d, world: %d\n",rank, size);

    MPI_Finalize();
}
EOF
ls -l
```

```
total 44
-rwxr-xr-x 1 root root 16208 Jul  2 18:16 a.out
-rwxr-xr-x 1 root root 16360 Jul  2 21:52 epccpiparallel
-rw-r--r-- 1 root root  2620 Jul  2 21:50 epccpiparallel.c
-rw-r--r-- 1 root root   330 Jul  2 21:54 hello_rank.c
drwxr-xr-x 1 root root  4096 Jul  1 13:21 sample_data
```

```
!mpicc hello_rank.c && ./a.out
```

```
➤ Hello: rank 0, world: 1
```

```
!mpirun --allow-run-as-root --oversubscribe -n 8 a.out
```

```
➤ Hello: rank 2, world: 8
Hello: rank 0, world: 8
Hello: rank 1, world: 8
Hello: rank 3, world: 8
Hello: rank 7, world: 8
Hello: rank 6, world: 8
Hello: rank 5, world: 8
Hello: rank 4, world: 8
```

✓ EPCC MPI example on PI calculation

```
%%sh
cat > epccpiparallel.c << EOF
/*
 * Example taken from: Message Passing Programming with MPI
 *
 * https://www.archer.ac.uk/training/course-material/2018/07/mpi-epcc/index.php
 * https://www.archer.ac.uk/training/course-material/2018/07/mpi-epcc/exercises/MPP-exercises.pdf
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#include <mpi.h>

#define N 840

int main(void)
{
    double pi, exactpi;

    int i;

    /* MPI variables */

    MPI_Comm comm;
    MPI_Status status;

    int rank, size, source, tag;

    /* Other variables */

    int istart, istop;
    double partialpi, recvpi;

    printf("Computing approximation to pi using N = %d\n", N);

    /* Initialise MPI and compute number of processes and local rank */

    comm = MPI_COMM_WORLD;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(comm, &size);
    MPI_Comm_rank(comm, &rank);

    printf("Hello from rank %d\n", rank);

    /* Now make sure output only comes from one process */

    if (rank == 0) printf("Running on %d process(es)\n", size);

    partialpi = 0.0;

    /*
     * Compute an approximation to pi using a simple series expansion for pi/4
     * Ensure each process computes a separate section of the summation
     * NOTE: here I assume that N is exactly divisible by the number of processes
     */

    istart = N/size * rank + 1;
    istop = istart + N/size - 1;

    printf("On rank %d istart = %d, istop = %d\n", rank, istart, istop);

    for (i=istart; i<=istop; i++)
    {
        partialpi = partialpi +
            1.0/( 1.0 + pow( (((double) i)-0.5)/((double) N), 2.0) );
    }

    printf("On rank %d partialpi = %f\n", rank, partialpi*4.0/((double) N));

    /*
     * Compute global value of pi by sending partial values to rank 0
     * NOTE: this would be more efficiently done using MPI_REDUCE
     */

    if (rank == 0)
    {
        /* Initialise pi to locally computed parial sum */
    }
}
```

```

    pi = partialpi;

    /* Add in contribution from other processes */

    for (source = 1; source < size; source++)
    {
        /* receive partialpi from *any* source and place value in recvpi */
        /* all messages are tagged as zero */

        tag = 0;

        MPI_Recv(&recvpi, 1, MPI_DOUBLE, MPI_ANY_SOURCE, tag, comm, &status);

        printf("rank 0 receiving from rank %d\n", status.MPI_SOURCE);

        /* add to running total */

        pi = pi + recvpi;
    }
}
else
{
    /* all other processes send their partial value to rank 0 */

    tag = 0;

    printf("rank %d sending to rank 0\n", rank);

    MPI_Ssend(&partialpi, 1, MPI_DOUBLE, 0, tag, comm);
}

pi = pi * 4.0/((double) N);

exactpi = 4.0*atan(1.0);

if (rank == 0)
{
    printf("pi = %f, %% error = %f\n", pi, fabs(100.0*(pi-exactpi)/exactpi));
}

MPI_Finalize();

return 0;
}
EOF

```

```
!mpicc epccpiparallel.c -o epccpiparallel -lm
```

```
!mpirun --allow-run-as-root --oversubscribe -n 8 epccpiparallel
```

```

🔄 Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Computing approximation to pi using N = 840
Hello from rank 1
Hello from rank 0
Hello from rank 3
Running on 8 process(es)
On rank 3 istart = 316, istop = 420
On rank 1 istart = 106, istop = 210
On rank 0 istart = 1, istop = 105
On rank 3 partialpi = 0.419508
On rank 1 partialpi = 0.482495
On rank 0 partialpi = 0.497420
rank 1 sending to rank 0
rank 3 sending to rank 0
rank 0 receiving from rank 1
rank 0 receiving from rank 3
Hello from rank 2
On rank 2 istart = 211, istop = 315
On rank 2 partialpi = 0.455168
rank 2 sending to rank 0
rank 0 receiving from rank 2
Hello from rank 5
On rank 5 istart = 526, istop = 630
On rank 5 partialpi = 0.339607
rank 5 sending to rank 0
rank 0 receiving from rank 5

```

```
Hello from rank 7
On rank 7 istart = 736, istop = 840
On rank 7 partialpi = 0.266273
rank 7 sending to rank 0
rank 0 receiving from rank 7
Hello from rank 4
On rank 4 istart = 421, istop = 525
On rank 4 partialpi = 0.379807
rank 4 sending to rank 0
rank 0 receiving from rank 4
Hello from rank 6
On rank 6 istart = 631, istop = 735
On rank 6 partialpi = 0.301316
rank 6 sending to rank 0
rank 0 receiving from rank 6
pi = 3.141593, % error = 0.000004
```